

Algoritmo Genético para el TSP

Asignatura: Algoritmos de Optimización

Alumno: Ricardo Germes Serrano

URL Repositorio GitHub de las 3 Prácticas: <https://github.com/RGS70/03MIAR-Proyecto/tree/main/PR%C3%81CTICAS>

URL GitHub (ipynb): https://github.com/RGS70/03MIAR-Proyecto/blob/main/PR%C3%81CTICAS/Algoritmo_Genetico_para_el_TSP_Ricardo_Germes_Serrano.ipynb

URL GitHub (pdf): https://github.com/RGS70/03MIAR-Proyecto/blob/main/PR%C3%81CTICAS/Algoritmo_Genetico_para_el_TSP_Ricardo_Germes_Serrano.pdf

Este notebook resuelve el esqueleto de la práctica de **Algoritmos Genéticos aplicados al TSP**, completando las funciones que estaban pendientes de implementar (`generar_poblacion` , `Evaluar_Poblacion` , `Cruzar` , `Descendencia` , `Factibilizar` , `Mutar` y `Seleccionar`), y se deja ya ejecutado de principio a fin sobre la instancia `swiss42.tsp` .

Instalación de librerías

```
In [1]: #Modulo de llamadas http para descargar ficheros
%pip install requests
import urllib.request

#Libreria del problema TSP: http://elib.zib.de/pub/mp-testdata/tsp/tsplib/tsplib.html
%pip install tsplib95
import tsplib95
```

```
Requirement already satisfied: requests in /usr/local/lib/python3.12/dist-packages (2.33.1)
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from request
s) (3.4.6)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests) (3.11)
Requirement already satisfied: urllib3<3,>=1.26 in /usr/local/lib/python3.12/dist-packages (from requests) (2.6.
3)
Requirement already satisfied: certifi>=2023.5.7 in /usr/local/lib/python3.12/dist-packages (from requests) (202
6.2.25)
Note: you may need to restart the kernel to use updated packages.
Requirement already satisfied: tsplib95 in /usr/local/lib/python3.12/dist-packages (0.7.1)
Requirement already satisfied: Click>=6.0 in /usr/local/lib/python3.12/dist-packages (from tsplib95) (8.3.1)
Requirement already satisfied: Deprecated~=1.2.9 in /usr/local/lib/python3.12/dist-packages (from tsplib95) (1.2
.18)
Requirement already satisfied: networkx~=2.1 in /usr/local/lib/python3.12/dist-packages (from tsplib95) (2.8.8)
Requirement already satisfied: tabulate~=0.8.7 in /usr/local/lib/python3.12/dist-packages (from tsplib95) (0.8.1
0)
Requirement already satisfied: wrapt<2,>=1.10 in /usr/local/lib/python3.12/dist-packages (from Deprecated~=1.2.9
->tsplib95) (1.17.3)
Note: you may need to restart the kernel to use updated packages.
```

```
In [2]: import random    #Libreria para generar numeros y listas aleatorias
import copy              #Permite hacer copias de objetos en python: listas, diccionarios,...
import gzip              #Para descomprimir el fichero .gz sin depender del comando de sistema "gzip"
import shutil
import socket
import os                #Para comprobar si el fichero ya existe en la carpeta local
```

Carga de datos del problema

```
In [3]: #Librerías y carga del problema

#http://elib.zib.de/pub/mp-testdata/tsp/tsplib/
#Documentacion :
# https://wwwproxy.iwr.uni-heidelberg.de/groups/comopt/software/TSPLIB95/tsp95.pdf
# https://tsplib95.readthedocs.io/usage.html
# https://tsplib95.readthedocs.io/modules.html#module-tsplib95.models

#Limitamos el tiempo de espera de conexión (evita que el notebook se quede
#"colgado" varios minutos si el servidor no responde)
socket.setdefaulttimeout(10)

file = "swiss42.tsp"

#Algunas redes (proxys/firewalls corporativos o de la universidad) no dejan
#conectar con el servidor de la Universidad de Heidelberg. Por eso probamos
```

```

#la fuente original y, si falla, comprobamos si el fichero ya está descargado
#en la carpeta local; solo si tampoco está ahí, caemos a un mirror en GitHub
#con los mismos datos (sin necesidad de descomprimir, ya viene en texto plano).
fuente_original = "http://comopt.ifi.uni-heidelberg.de/software/TSPLIB95/tsp/swiss42.tsp.gz"
fuente_alternativa = "https://github.com/mastqe/tsplib/raw/refs/heads/master/swiss42.tsp"

descargado = False

#1er intento: fuente original (fichero comprimido en .gz)
try:
    urllib.request.urlretrieve(fuente_original, file + '.gz')
    with gzip.open(file + '.gz', 'rb') as f_in:
        with open(file, 'wb') as f_out:
            shutil.copyfileobj(f_in, f_out)
    descargado = True
    print("Datos descargados desde la fuente original (Universidad de Heidelberg).")
except Exception as e:
    print("Aviso: no se ha podido descargar desde la fuente original ->", e)

#2o intento: comprobamos si el fichero ya existe en la carpeta local
if not descargado and os.path.exists(file):
    descargado = True
    print(f"El fichero '{file}' ya existe en la carpeta local. Se usará ese fichero.")

#3er intento (fallback): mirror alternativo en GitHub
if not descargado:
    try:
        urllib.request.urlretrieve(fuente_alternativa, file)
        descargado = True
        print("Datos descargados desde el mirror alternativo de GitHub.")
    except Exception as e:
        print("Aviso: no se ha podido descargar desde el mirror alternativo ->", e)

if not descargado:
    raise RuntimeError("No ha sido posible descargar el fichero swiss42.tsp desde ninguna fuente.")

#Objeto de tsplib95 para nuestro problema problema
problem = tsplib95.load(file)

#Nodos
Nodos = list(problem.get_nodes())

#Aristas
Aristas = list(problem.get_edges())
print("Número de nodos:", len(Nodos))

#Coordenadas(si estan disponibles en el fichero)
problem.get_display(1)

#Distancia
problem.get_weight(1, 2)

```

Aviso: no se ha podido descargar desde la fuente original -> HTTP Error 403: Forbidden
 Datos descargados desde el mirror alternativo de GitHub.
 Número de nodos: 42

Out[3]: 34

Funciones de la Actividad Guiada 3

```

In [4]: #Funciones de la Actividad Guiada 3
#Se genera una solucion aleatoria con comienzo en el nodo 0
def crear_solucion(Nodos):
    solucion = [Nodos[0]]
    for n in Nodos[1:]:
        solucion = solucion + [random.choice(list(set(Nodos) - set(solucion)))]
    return solucion

#Devuelve la distancia entre dos nodos
def distancia(a,b, problem):
    return problem.get_weight(a,b)

#Devuelve la distancia total de una trayectoria/solucion
def distancia_total(solucion, problem):
    distancia_total = 0
    for i in range(len(solucion)-1):
        distancia_total += distancia(solucion[i],solucion[i+1], problem)
    return distancia_total + distancia(solucion[len(solucion)-1],solucion[0], problem)

```

Funciones Auxiliares

```
In [5]: #Genera una poblacion inicial de soluciones de tamaño N.
# Puede ser válida la solución aleatoria de la AG3: crear_solucion(Nodos)
def generar_poblacion(Nodos,N):
    poblacion = [crear_solucion(Nodos) for _ in range(N)]
    return poblacion
```

```
In [6]: #Evalua la población y devuelve el mejor individuo
def Evaluar_Poblacion(poblacion, problem):
    mejor_solucion = poblacion[0]
    mejor_distancia = distancia_total(poblacion[0], problem)

    for solucion in poblacion[1:]:
        distancia_actual = distancia_total(solucion, problem)
        if distancia_actual < mejor_distancia:
            mejor_solucion = solucion
            mejor_distancia = distancia_actual

    return (mejor_solucion, mejor_distancia)
```

```
In [7]: #Funcion de cruce. Recibe una poblacion(lista de soluciones) y devuelve la población ampliada con los hijos.
# Todos los individuos de la población son seleccionados para el cruce(si la población es par)
# Podría aplicarse un proceso previo de selección para elegir los individuos que se desea cruzar.
def Cruzar(poblacion,mutacion, problem):
    #Barajamos la población para emparejar individuos al azar en cada generación
    #(evita cruzar siempre a los mismos vecinos de la lista)
    poblacion_a_cruzar = copy.deepcopy(poblacion)
    random.shuffle(poblacion_a_cruzar)

    hijos = []
    for i in range(0, len(poblacion_a_cruzar) - 1, 2):
        padres = (poblacion_a_cruzar[i], poblacion_a_cruzar[i+1])
        hijos += Descendencia(padres, problem, mutacion)

    #Devolvemos la población ampliada: padres originales + hijos generados
    return poblacion + hijos
```

```
In [8]: #Funcion para generar hijos a partir de 2 padres:
# Se elige el metodo de 1-punto de corte pero es posible usar otros n-puntos, uniforme, dependiendo del problema
def Descendencia(padres, problem,mutacion):
    padre1, padre2 = padres
    n = len(padre1)

    #Elegimos un punto de corte (evitamos la posición 0, que es siempre el nodo de inicio)
    punto_corte = random.randint(1, n-1)

    #Hijo 1: primera parte de padre1 + segunda parte de padre2
    hijo1 = padre1[:punto_corte] + padre2[punto_corte:]
    #Hijo 2: primera parte de padre2 + segunda parte de padre1
    hijo2 = padre2[:punto_corte] + padre1[punto_corte:]

    #Los hijos generados así pueden tener nodos repetidos/faltantes: los reparamos
    hijo1 = Factibilizar(hijo1, problem)
    hijo2 = Factibilizar(hijo2, problem)

    #Aplicamos el operador de mutación a cada hijo
    hijo1 = Mutar(hijo1, mutacion)
    hijo2 = Mutar(hijo2, mutacion)

    return [hijo1, hijo2]
```

```
In [9]: #Para el operador de cruce 1-punto los hijos generados no son soluciones(algunos nodos se repiten y otros no es)
def Factibilizar(solucion,problem):
    Nodos = list(problem.get_nodos())

    #Nodos que faltan por visitar en la solución
    faltantes = [n for n in Nodos if n not in solucion]
    random.shuffle(faltantes)

    vistos = set()
    solucion_factible = []
    for nodo in solucion:
        if nodo not in vistos:
            solucion_factible.append(nodo)
            vistos.add(nodo)
        else:
            #Si el nodo ya apareció antes (repetido), lo sustituimos por uno de los que faltan
            nuevo_nodo = faltantes.pop()
            solucion_factible.append(nuevo_nodo)
            vistos.add(nuevo_nodo)
```

```
return solucion_factible
```

```
In [10]: #Funcion de mutación. Se eligen dos nodos y se intercambia. Se podrian añadir otros operadores
# Se hace mutaciones mutacion% de las veces
def Mutar(solucion,mutacion):
    if random.random() <= mutacion:
        #Elegimos dos posiciones al azar (evitamos la posición 0, nodo de inicio fijo)
        i, j = random.sample(range(1, len(solucion)), 2)
        solucion[i], solucion[j] = solucion[j], solucion[i]
    return solucion
```

```
In [11]: #Funcion de seleccion de la población. Recibe como parametro una poblacion y
# devuelve una poblacion a la que se ha eliminado individuos poco aptos(fitness alto) y para mantener una pobla
#Se tiene en cuenta el porcentaje elitismo pasado como parametro
# Para los individuos que no son de la elite podríamos usar una selección de ruleta(proporcional a su fitness)
def Seleccionar(problem,poblacion, N, elitismo):
    #Ordenamos la población de mejor a peor (menor distancia = mejor)
    poblacion_ordenada = sorted(poblacion, key=lambda solucion: distancia_total(solucion, problem))

    #Nos quedamos siempre con la élite (los mejores individuos)
    n_elite = int(N * elitismo)
    elite = poblacion_ordenada[:n_elite]

    #El resto hasta completar N se elige mediante selección de ruleta, proporcional al fitness
    #(a menor distancia, mayor fitness = mayor probabilidad de ser seleccionado)
    resto_poblacion = poblacion_ordenada[n_elite:]
    distancias = [distancia_total(s, problem) for s in resto_poblacion]
    fitness = [1 / d for d in distancias]
    total_fitness = sum(fitness)
    probabilidades = [f / total_fitness for f in fitness]

    n_restantes = N - n_elite
    seleccionados = random.choices(resto_poblacion, weights=probabilidades, k=n_restantes) if resto_poblacion else

    return elite + seleccionados
```

Proceso Principal

```
In [12]: #Funcion principal del algoritmo genetico
#####3
def algoritmo_genetico(problem=problem,N=100,mutacion=.15,elitismo=.1,generaciones=100):
    # problem = datos del problema
    # N = Tamaño de la población
    # mutacion = probabilidad de una mutación
    # elitismo = porcion de la mejor poblacion a mantener
    # generaciones = nº de generaciones a generar para finalizar

    #Genera la poblacion inicial
    Nodos = list(problem.get_nodes())
    poblacion = generar_poblacion(Nodos,N)

    #Inicializamos valores para la mejor solucion
    (mejor_solucion, mejor_distancia) = Evaluar_Poblacion(poblacion, problem)

    #Condicion de parada
    parar = False
    n=0
    #Iniciamos el ciclo de generaciones
    while(parar == False) :

        #Cruce de la poblacion(incluye mutación)
        poblacion = Cruzar(poblacion, mutacion, problem)

        #Seleccionamos la población
        poblacion = Seleccionar(problem,poblacion, N, elitismo)

        #Evaluamos la nueva población
        (mejor_solucion, mejor_distancia) = Evaluar_Poblacion(poblacion, problem)

        print("Generacion #", n, "\nLa mejor solución es:" , mejor_solucion, "\ncon distancia " , mejor_distancia,

        #Numero de generaciones. Criterio de parada
        if n==generaciones:
            parar = True
            n +=1

    return mejor_solucion
```

```
sol = algoritmo_genetico(problem=problem,N=500,mutacion=.3,elitismo=.40,generaciones=250)
```

Generacion # 0

La mejor solución es: [0, 1, 20, 13, 3, 8, 12, 26, 22, 33, 27, 2, 10, 29, 16, 7, 35, 21, 24, 23, 4, 36, 14, 25, 39, 40, 41, 9, 38, 30, 31, 37, 18, 5, 6, 19, 17, 11, 15, 34, 32, 28]

con distancia 3828

Generacion # 1

La mejor solución es: [0, 1, 20, 13, 3, 8, 12, 26, 22, 33, 27, 2, 10, 29, 16, 7, 35, 21, 24, 23, 4, 36, 14, 25, 39, 40, 41, 9, 38, 30, 31, 37, 18, 5, 6, 19, 17, 11, 15, 34, 32, 28]

con distancia 3828

Generacion # 2

La mejor solución es: [0, 20, 33, 35, 14, 37, 4, 5, 15, 36, 25, 10, 8, 30, 2, 3, 23, 1, 29, 40, 11, 34, 16, 13, 17, 32, 7, 6, 22, 24, 9, 38, 27, 28, 39, 21, 19, 18, 31, 12, 41, 26]

con distancia 3821

Generacion # 3

La mejor solución es: [0, 17, 7, 5, 10, 11, 25, 41, 4, 6, 12, 13, 34, 15, 14, 18, 23, 40, 30, 39, 9, 2, 31, 16, 1, 3, 32, 28, 37, 20, 27, 24, 38, 35, 21, 29, 8, 22, 33, 26, 19, 36]

con distancia 3778

Generacion # 4

La mejor solución es: [0, 17, 7, 5, 10, 11, 25, 41, 4, 6, 12, 13, 34, 15, 14, 18, 23, 40, 30, 39, 9, 2, 31, 16, 1, 3, 32, 28, 37, 20, 27, 24, 38, 35, 21, 29, 8, 22, 33, 26, 19, 36]

con distancia 3778

Generacion # 5

La mejor solución es: [0, 13, 30, 34, 29, 39, 24, 8, 23, 3, 6, 16, 26, 11, 18, 7, 33, 15, 5, 20, 35, 31, 17, 27, 10, 19, 36, 14, 37, 12, 9, 40, 21, 32, 28, 1, 41, 4, 25, 2, 38, 22]

con distancia 3600

Generacion # 6

La mejor solución es: [0, 13, 30, 34, 29, 39, 24, 8, 23, 3, 6, 16, 26, 11, 18, 7, 33, 15, 5, 20, 35, 31, 17, 27, 10, 19, 36, 14, 37, 12, 9, 40, 21, 32, 28, 1, 41, 4, 25, 2, 38, 22]

con distancia 3600

Generacion # 7

La mejor solución es: [0, 13, 30, 34, 29, 39, 24, 8, 23, 3, 6, 16, 26, 11, 18, 7, 33, 15, 5, 20, 35, 31, 17, 27, 10, 19, 36, 14, 37, 12, 9, 40, 21, 32, 28, 1, 41, 4, 25, 2, 38, 22]

con distancia 3600

Generacion # 8

La mejor solución es: [0, 6, 5, 12, 11, 29, 8, 26, 31, 38, 39, 17, 36, 33, 20, 35, 13, 14, 32, 28, 27, 18, 9, 41, 1, 7, 19, 4, 10, 3, 21, 22, 40, 30, 24, 23, 25, 2, 16, 37, 15, 34]

con distancia 3424

Generacion # 9

La mejor solución es: [0, 6, 5, 12, 11, 29, 8, 26, 31, 38, 39, 17, 36, 33, 20, 35, 13, 14, 32, 28, 27, 18, 9, 41, 1, 7, 19, 4, 10, 3, 21, 22, 40, 30, 24, 23, 25, 2, 16, 37, 15, 34]

con distancia 3424

Generacion # 10

La mejor solución es: [0, 20, 33, 35, 39, 30, 36, 31, 16, 37, 17, 18, 24, 21, 26, 25, 10, 28, 11, 1, 27, 3, 6, 1, 9, 5, 13, 8, 14, 15, 7, 4, 32, 2, 41, 9, 23, 22, 12, 40, 29, 38, 34]

con distancia 3387

Generacion # 11

La mejor solución es: [0, 20, 33, 34, 29, 39, 24, 8, 23, 3, 6, 41, 26, 11, 18, 7, 1, 15, 5, 2, 35, 37, 31, 36, 1, 3, 17, 19, 22, 38, 4, 14, 16, 25, 12, 10, 9, 30, 27, 28, 40, 21, 32]

con distancia 3241

Generacion # 12

La mejor solución es: [0, 20, 33, 34, 29, 39, 24, 8, 23, 3, 6, 41, 26, 11, 18, 7, 1, 15, 5, 2, 35, 37, 31, 36, 1, 3, 17, 19, 22, 38, 4, 14, 16, 25, 12, 10, 9, 21, 40, 28, 30, 32, 27]

con distancia 3118

Generacion # 13

La mejor solución es: [0, 20, 33, 34, 29, 39, 24, 8, 23, 3, 6, 41, 26, 11, 18, 7, 1, 15, 5, 2, 35, 37, 31, 36, 1, 3, 17, 19, 22, 38, 4, 14, 16, 25, 12, 10, 9, 21, 40, 28, 30, 32, 27]

con distancia 3118

Generacion # 14

La mejor solución es: [0, 20, 33, 34, 29, 39, 24, 8, 23, 3, 6, 41, 26, 11, 18, 7, 1, 15, 5, 2, 35, 37, 31, 36, 1, 3, 17, 19, 22, 38, 4, 14, 16, 25, 12, 10, 9, 21, 40, 28, 30, 32, 27]

con distancia 3118

Generacion # 15

La mejor solución es: [0, 20, 33, 35, 16, 15, 37, 5, 25, 41, 18, 4, 38, 22, 21, 8, 9, 39, 23, 30, 13, 19, 6, 26, 27, 1, 32, 31, 7, 3, 36, 17, 14, 34, 40, 10, 11, 12, 29, 28, 24, 2]

con distancia 3058

Generacion # 16

La mejor solución es: [0, 20, 33, 35, 16, 15, 37, 5, 25, 41, 18, 4, 38, 22, 21, 8, 9, 39, 23, 30, 13, 19, 6, 26, 27, 1, 32, 31, 7, 3, 36, 17, 14, 34, 40, 10, 11, 12, 29, 28, 24, 2]
con distancia 3058

Generacion # 17

La mejor solución es: [0, 20, 33, 35, 16, 15, 37, 5, 25, 41, 18, 4, 38, 22, 21, 8, 9, 39, 23, 30, 13, 19, 6, 26, 27, 1, 32, 31, 7, 3, 36, 17, 14, 34, 40, 10, 11, 12, 29, 28, 24, 2]
con distancia 3058

Generacion # 18

La mejor solución es: [0, 20, 33, 35, 16, 15, 37, 5, 25, 41, 18, 4, 38, 22, 21, 8, 9, 39, 23, 30, 13, 19, 6, 26, 27, 1, 32, 31, 7, 3, 36, 17, 14, 34, 40, 10, 11, 12, 29, 28, 24, 2]
con distancia 3058

Generacion # 19

La mejor solución es: [0, 20, 33, 35, 16, 15, 37, 5, 25, 41, 18, 4, 38, 22, 21, 8, 9, 39, 23, 30, 13, 19, 6, 26, 27, 1, 32, 31, 7, 3, 36, 17, 14, 34, 40, 10, 11, 12, 29, 28, 24, 2]
con distancia 3058

Generacion # 20

La mejor solución es: [0, 20, 33, 35, 16, 15, 37, 5, 25, 41, 18, 4, 38, 22, 21, 8, 9, 39, 23, 30, 29, 24, 6, 26, 27, 1, 13, 14, 19, 12, 11, 40, 10, 32, 28, 3, 7, 34, 31, 2, 36, 17]
con distancia 3039

Generacion # 21

La mejor solución es: [0, 20, 33, 35, 16, 15, 37, 5, 25, 41, 18, 4, 38, 22, 21, 8, 9, 39, 23, 30, 13, 19, 6, 26, 27, 1, 32, 31, 7, 3, 36, 17, 14, 34, 40, 10, 11, 12, 29, 24, 28, 2]
con distancia 3006

Generacion # 22

La mejor solución es: [0, 20, 33, 35, 16, 15, 37, 5, 25, 41, 18, 4, 38, 22, 21, 8, 9, 39, 23, 30, 13, 19, 6, 26, 27, 1, 32, 31, 7, 3, 36, 17, 14, 34, 40, 10, 11, 12, 29, 24, 28, 2]
con distancia 3006

Generacion # 23

La mejor solución es: [0, 20, 33, 34, 29, 39, 24, 8, 25, 41, 18, 6, 26, 23, 3, 7, 1, 19, 15, 2, 5, 35, 31, 36, 1, 3, 17, 37, 22, 38, 4, 14, 16, 11, 12, 10, 9, 21, 40, 28, 30, 27, 32]
con distancia 2965

Generacion # 24

La mejor solución es: [0, 20, 33, 34, 29, 39, 24, 8, 25, 41, 18, 6, 26, 23, 3, 7, 1, 19, 15, 2, 5, 35, 31, 36, 1, 3, 17, 37, 22, 38, 4, 14, 16, 11, 12, 10, 9, 21, 40, 28, 30, 27, 32]
con distancia 2965

Generacion # 25

La mejor solución es: [0, 20, 33, 35, 16, 15, 37, 5, 25, 41, 18, 4, 38, 22, 21, 8, 9, 39, 23, 30, 29, 24, 40, 26, 27, 1, 32, 34, 28, 10, 13, 6, 3, 17, 7, 14, 19, 36, 31, 2, 11, 12]
con distancia 2848

Generacion # 26

La mejor solución es: [0, 20, 33, 35, 16, 15, 37, 5, 25, 41, 18, 4, 38, 22, 21, 8, 9, 39, 23, 30, 29, 24, 40, 26, 27, 1, 32, 34, 28, 10, 13, 6, 3, 17, 7, 14, 19, 36, 31, 2, 11, 12]
con distancia 2848

Generacion # 27

La mejor solución es: [0, 20, 33, 35, 16, 15, 37, 5, 25, 41, 18, 4, 38, 22, 21, 8, 9, 39, 23, 30, 29, 24, 40, 26, 27, 1, 32, 34, 28, 10, 13, 6, 3, 17, 7, 14, 19, 36, 31, 2, 11, 12]
con distancia 2848

Generacion # 28

La mejor solución es: [0, 20, 33, 35, 16, 15, 37, 5, 25, 41, 18, 4, 38, 22, 21, 23, 9, 39, 8, 30, 13, 19, 6, 2, 3, 10, 29, 26, 11, 12, 1, 17, 36, 7, 14, 32, 34, 40, 24, 28, 27, 31]
con distancia 2842

Generacion # 29

La mejor solución es: [0, 20, 33, 35, 16, 15, 37, 5, 25, 41, 18, 4, 38, 22, 21, 23, 9, 39, 8, 30, 13, 19, 6, 2, 3, 10, 29, 26, 11, 12, 1, 17, 36, 7, 14, 32, 34, 40, 24, 28, 27, 31]
con distancia 2842

Generacion # 30

La mejor solución es: [0, 20, 33, 35, 16, 15, 37, 5, 25, 41, 18, 4, 38, 22, 21, 23, 9, 39, 8, 30, 13, 19, 6, 2, 3, 10, 29, 26, 11, 12, 1, 17, 36, 7, 14, 32, 34, 40, 24, 28, 27, 31]
con distancia 2842

Generacion # 31

La mejor solución es: [0, 20, 33, 35, 16, 15, 37, 5, 25, 41, 18, 4, 38, 22, 21, 8, 9, 39, 23, 30, 13, 19, 6, 2, 32, 1, 26, 14, 10, 12, 11, 40, 24, 29, 28, 3, 36, 31, 34, 27, 7, 17]
con distancia 2816

Generacion # 32

La mejor solución es: [0, 20, 33, 35, 16, 15, 37, 5, 25, 41, 18, 4, 38, 22, 21, 8, 9, 39, 23, 30, 13, 19, 6, 2, 32, 1, 26, 14, 10, 12, 11, 40, 24, 29, 28, 3, 36, 31, 34, 27, 7, 17]

con distancia 2816

Generacion # 33

La mejor solución es: [0, 20, 33, 35, 16, 15, 37, 5, 25, 41, 18, 4, 38, 22, 21, 40, 9, 39, 24, 30, 29, 23, 6, 26, 27, 1, 13, 14, 19, 12, 11, 8, 34, 32, 28, 36, 7, 17, 31, 2, 3, 10]

con distancia 2768

Generacion # 34

La mejor solución es: [0, 20, 33, 35, 16, 15, 37, 5, 25, 41, 18, 4, 38, 22, 21, 40, 9, 39, 24, 30, 29, 23, 6, 26, 27, 1, 13, 14, 19, 12, 11, 8, 34, 32, 28, 36, 7, 17, 31, 2, 3, 10]

con distancia 2768

Generacion # 35

La mejor solución es: [0, 20, 33, 35, 16, 15, 37, 5, 25, 41, 18, 4, 38, 22, 21, 8, 9, 39, 23, 30, 13, 19, 6, 2, 32, 1, 26, 14, 10, 12, 11, 40, 24, 29, 28, 3, 7, 34, 31, 17, 36, 27]

con distancia 2767

Generacion # 36

La mejor solución es: [0, 20, 33, 35, 16, 15, 37, 5, 25, 41, 18, 4, 38, 22, 21, 40, 9, 39, 23, 30, 29, 24, 6, 26, 27, 1, 13, 14, 19, 12, 11, 8, 10, 32, 28, 3, 7, 34, 36, 31, 17, 2]

con distancia 2753

Generacion # 37

La mejor solución es: [0, 20, 33, 35, 16, 15, 37, 5, 25, 41, 18, 4, 38, 22, 21, 40, 24, 39, 9, 23, 29, 8, 6, 26, 27, 1, 13, 14, 19, 12, 11, 30, 10, 32, 28, 3, 7, 34, 31, 36, 17, 2]

con distancia 2590

Generacion # 38

La mejor solución es: [0, 20, 33, 35, 16, 15, 37, 5, 25, 41, 18, 4, 38, 22, 21, 40, 24, 39, 9, 23, 29, 8, 6, 26, 27, 1, 13, 14, 19, 12, 11, 30, 10, 32, 28, 3, 7, 34, 31, 36, 17, 2]

con distancia 2590

Generacion # 39

La mejor solución es: [0, 34, 33, 35, 16, 15, 37, 5, 25, 41, 18, 4, 38, 22, 21, 40, 24, 39, 9, 23, 29, 8, 6, 26, 27, 1, 13, 14, 19, 12, 11, 30, 10, 32, 28, 3, 7, 20, 31, 36, 17, 2]

con distancia 2563

Generacion # 40

La mejor solución es: [0, 34, 33, 35, 16, 15, 37, 5, 25, 41, 18, 4, 38, 22, 21, 40, 24, 39, 9, 23, 29, 8, 6, 26, 27, 1, 13, 14, 19, 12, 11, 30, 10, 32, 28, 3, 7, 20, 31, 36, 17, 2]

con distancia 2563

Generacion # 41

La mejor solución es: [0, 20, 33, 35, 16, 15, 37, 5, 25, 41, 18, 4, 38, 22, 21, 40, 24, 39, 23, 30, 29, 10, 6, 2, 6, 27, 1, 13, 14, 19, 12, 11, 9, 8, 32, 28, 2, 31, 17, 36, 34, 3, 7]

con distancia 2537

Generacion # 42

La mejor solución es: [0, 20, 33, 35, 16, 15, 37, 5, 25, 41, 18, 4, 38, 22, 21, 40, 24, 39, 23, 30, 29, 10, 6, 2, 6, 27, 1, 13, 14, 19, 12, 11, 9, 8, 32, 28, 2, 31, 17, 36, 34, 3, 7]

con distancia 2537

Generacion # 43

La mejor solución es: [0, 20, 33, 35, 16, 15, 37, 5, 25, 41, 18, 4, 38, 22, 21, 40, 24, 39, 23, 30, 29, 10, 6, 2, 6, 27, 1, 13, 14, 19, 12, 11, 9, 8, 32, 28, 2, 31, 17, 36, 34, 3, 7]

con distancia 2537

Generacion # 44

La mejor solución es: [0, 20, 33, 35, 16, 15, 37, 5, 25, 41, 18, 4, 38, 22, 21, 40, 24, 39, 9, 23, 29, 8, 6, 26, 27, 1, 13, 14, 19, 12, 11, 30, 10, 2, 28, 3, 36, 34, 32, 31, 17, 7]

con distancia 2528

Generacion # 45

La mejor solución es: [0, 20, 33, 35, 16, 15, 37, 5, 25, 41, 18, 4, 38, 22, 21, 40, 24, 39, 9, 23, 29, 30, 6, 26, 27, 1, 13, 14, 19, 12, 11, 8, 10, 32, 28, 3, 7, 34, 31, 36, 17, 2]

con distancia 2466

Generacion # 46

La mejor solución es: [0, 20, 33, 35, 16, 15, 37, 5, 25, 41, 18, 4, 38, 22, 21, 40, 24, 39, 9, 23, 29, 30, 6, 26, 27, 1, 13, 14, 19, 12, 11, 8, 10, 32, 28, 3, 7, 34, 31, 36, 17, 2]

con distancia 2466

Generacion # 47

La mejor solución es: [0, 20, 33, 35, 16, 15, 37, 5, 25, 41, 18, 4, 38, 22, 21, 40, 24, 39, 9, 23, 29, 30, 6, 26, 27, 1, 13, 14, 19, 12, 11, 8, 10, 32, 28, 3, 7, 34, 31, 36, 17, 2]

con distancia 2466

Generacion # 48

La mejor solución es: [0, 20, 33, 35, 16, 15, 37, 5, 25, 41, 18, 4, 38, 22, 21, 40, 24, 39, 9, 23, 29, 30, 6, 26, 27, 1, 13, 14, 19, 12, 11, 8, 10, 32, 28, 3, 7, 34, 31, 36, 17, 2]

con distancia 2466

Generacion # 49

La mejor solución es: [0, 20, 33, 35, 16, 15, 37, 5, 25, 41, 18, 4, 38, 22, 21, 40, 24, 39, 9, 23, 29, 30, 6, 26, 27, 1, 13, 14, 19, 12, 11, 8, 10, 32, 28, 2, 31, 17, 36, 34, 3, 7]

con distancia 2459

Generacion # 50

La mejor solución es: [0, 20, 33, 35, 16, 15, 37, 5, 25, 41, 18, 4, 38, 22, 21, 40, 24, 39, 9, 23, 29, 8, 6, 3, 27, 1, 13, 14, 19, 12, 11, 10, 2, 30, 28, 26, 7, 32, 36, 17, 31, 34]

con distancia 2434

Generacion # 51

La mejor solución es: [0, 20, 33, 35, 16, 15, 37, 5, 25, 41, 18, 4, 38, 22, 21, 40, 24, 39, 9, 23, 29, 30, 6, 26, 27, 1, 13, 14, 19, 12, 11, 10, 8, 32, 28, 2, 31, 17, 36, 34, 3, 7]

con distancia 2420

Generacion # 52

La mejor solución es: [0, 20, 33, 35, 16, 15, 37, 5, 25, 41, 18, 4, 38, 22, 39, 40, 24, 21, 9, 23, 29, 30, 6, 26, 27, 1, 13, 14, 19, 12, 11, 8, 10, 2, 28, 3, 7, 34, 17, 31, 36, 32]

con distancia 2407

Generacion # 53

La mejor solución es: [0, 20, 33, 35, 16, 15, 37, 5, 25, 41, 18, 4, 38, 22, 39, 40, 24, 21, 9, 23, 29, 30, 6, 26, 27, 1, 13, 14, 19, 12, 11, 8, 10, 2, 28, 3, 7, 34, 17, 31, 36, 32]

con distancia 2407

Generacion # 54

La mejor solución es: [0, 20, 33, 34, 16, 15, 37, 5, 25, 41, 18, 4, 38, 22, 21, 40, 24, 39, 9, 23, 29, 30, 6, 26, 27, 1, 13, 14, 19, 12, 11, 8, 10, 32, 28, 2, 3, 35, 36, 31, 17, 7]

con distancia 2269

Generacion # 55

La mejor solución es: [0, 20, 33, 34, 16, 15, 37, 5, 25, 41, 18, 4, 38, 22, 21, 40, 24, 39, 9, 23, 29, 30, 6, 26, 27, 1, 13, 14, 19, 12, 11, 8, 10, 32, 28, 2, 3, 35, 36, 31, 17, 7]

con distancia 2269

Generacion # 56

La mejor solución es: [0, 20, 33, 34, 16, 15, 37, 5, 25, 41, 18, 4, 38, 22, 21, 40, 24, 39, 9, 23, 29, 30, 2, 1, 27, 26, 13, 14, 19, 12, 11, 8, 10, 6, 28, 3, 31, 17, 36, 35, 7, 32]

con distancia 2266

Generacion # 57

La mejor solución es: [0, 20, 33, 34, 16, 15, 37, 5, 25, 41, 18, 4, 38, 22, 21, 40, 24, 39, 9, 23, 29, 30, 2, 1, 27, 26, 13, 14, 19, 12, 11, 8, 10, 6, 28, 3, 31, 17, 36, 35, 7, 32]

con distancia 2266

Generacion # 58

La mejor solución es: [0, 20, 33, 34, 16, 15, 37, 5, 25, 41, 18, 4, 38, 22, 21, 40, 24, 39, 9, 23, 29, 30, 2, 1, 27, 26, 13, 14, 19, 12, 11, 8, 10, 6, 28, 3, 31, 17, 36, 35, 7, 32]

con distancia 2266

Generacion # 59

La mejor solución es: [0, 20, 33, 34, 16, 15, 37, 5, 25, 41, 18, 4, 38, 22, 21, 40, 24, 39, 9, 23, 29, 30, 2, 1, 27, 26, 13, 14, 19, 12, 11, 8, 10, 6, 28, 3, 31, 17, 36, 35, 7, 32]

con distancia 2266

Generacion # 60

La mejor solución es: [0, 20, 33, 34, 16, 15, 37, 5, 25, 41, 18, 4, 38, 22, 21, 40, 24, 39, 9, 23, 29, 30, 2, 1, 6, 26, 13, 14, 19, 12, 11, 10, 8, 32, 28, 27, 31, 17, 36, 35, 3, 7]

con distancia 2223

Generacion # 61

La mejor solución es: [0, 20, 33, 34, 16, 15, 37, 5, 25, 41, 18, 4, 38, 22, 21, 40, 24, 39, 9, 23, 29, 30, 2, 1, 6, 26, 13, 14, 19, 12, 11, 10, 8, 32, 28, 27, 31, 17, 36, 35, 3, 7]

con distancia 2223

Generacion # 62

La mejor solución es: [0, 20, 33, 34, 16, 15, 37, 5, 25, 41, 18, 4, 38, 22, 21, 40, 24, 39, 9, 23, 29, 30, 2, 1, 6, 26, 13, 14, 19, 12, 11, 10, 8, 32, 28, 27, 31, 17, 36, 35, 3, 7]

con distancia 2223

Generacion # 63

La mejor solución es: [0, 20, 33, 34, 16, 15, 37, 5, 25, 41, 18, 4, 38, 22, 21, 40, 24, 39, 9, 23, 29, 30, 2, 1, 27, 26, 13, 14, 19, 12, 11, 8, 10, 6, 28, 3, 32, 35, 36, 31, 17, 7]

con distancia 2208

Generacion # 96

La mejor solución es: [0, 34, 33, 20, 37, 15, 16, 5, 25, 41, 18, 4, 38, 22, 39, 40, 24, 21, 23, 9, 29, 30, 2, 1, 6, 26, 13, 14, 19, 12, 11, 10, 8, 28, 27, 3, 7, 17, 36, 35, 31, 32]

con distancia 1979

Generacion # 97

La mejor solución es: [0, 34, 33, 20, 37, 15, 16, 14, 25, 10, 18, 4, 38, 22, 21, 40, 24, 39, 9, 23, 29, 30, 2, 1, 6, 26, 13, 5, 19, 12, 11, 41, 8, 28, 27, 3, 7, 17, 36, 35, 31, 32]

con distancia 1965

Generacion # 98

La mejor solución es: [0, 34, 33, 20, 37, 15, 16, 14, 25, 10, 18, 4, 38, 22, 21, 40, 24, 39, 9, 23, 29, 30, 2, 1, 6, 26, 13, 5, 19, 12, 11, 41, 8, 28, 27, 3, 7, 17, 36, 35, 31, 32]

con distancia 1965

Generacion # 99

La mejor solución es: [0, 20, 33, 34, 37, 15, 16, 14, 25, 41, 18, 4, 38, 22, 39, 40, 24, 21, 9, 23, 29, 30, 2, 1, 6, 26, 5, 13, 19, 12, 11, 10, 8, 28, 27, 3, 32, 35, 36, 31, 17, 7]

con distancia 1951

Generacion # 100

La mejor solución es: [0, 20, 33, 34, 37, 15, 16, 14, 25, 41, 18, 4, 38, 22, 39, 40, 24, 21, 9, 23, 29, 30, 2, 1, 6, 26, 5, 13, 19, 12, 11, 10, 8, 28, 27, 3, 32, 35, 36, 31, 17, 7]

con distancia 1951

Generacion # 101

La mejor solución es: [0, 20, 33, 34, 37, 15, 16, 14, 25, 41, 18, 4, 38, 22, 39, 40, 24, 21, 9, 23, 29, 30, 2, 1, 6, 26, 5, 13, 19, 12, 11, 10, 8, 28, 27, 3, 32, 35, 36, 31, 17, 7]

con distancia 1951

Generacion # 102

La mejor solución es: [0, 34, 33, 20, 37, 15, 16, 13, 25, 10, 18, 4, 38, 22, 39, 40, 24, 21, 23, 9, 29, 30, 2, 1, 6, 26, 5, 14, 19, 12, 11, 41, 8, 28, 27, 3, 32, 35, 36, 31, 17, 7]

con distancia 1946

Generacion # 103

La mejor solución es: [0, 20, 33, 34, 37, 15, 16, 14, 25, 41, 18, 4, 38, 22, 39, 40, 24, 21, 23, 9, 29, 30, 2, 1, 6, 26, 5, 13, 19, 12, 11, 10, 8, 28, 27, 3, 32, 35, 36, 17, 31, 7]

con distancia 1942

Generacion # 104

La mejor solución es: [0, 34, 33, 20, 37, 15, 16, 14, 25, 41, 18, 28, 38, 22, 39, 40, 24, 21, 23, 9, 29, 30, 4, 1, 6, 26, 5, 13, 19, 12, 11, 10, 8, 3, 2, 27, 32, 35, 36, 31, 17, 7]

con distancia 1921

Generacion # 105

La mejor solución es: [0, 34, 33, 20, 37, 15, 16, 14, 25, 41, 18, 4, 38, 22, 39, 40, 24, 21, 23, 9, 29, 30, 28, 1, 6, 26, 5, 13, 19, 12, 11, 10, 8, 3, 2, 27, 32, 35, 36, 31, 17, 7]

con distancia 1917

Generacion # 106

La mejor solución es: [0, 34, 33, 20, 37, 15, 16, 14, 25, 41, 18, 4, 38, 22, 39, 40, 24, 21, 23, 9, 29, 30, 28, 1, 6, 26, 5, 13, 19, 12, 11, 10, 8, 3, 2, 27, 32, 35, 36, 31, 17, 7]

con distancia 1917

Generacion # 107

La mejor solución es: [0, 34, 33, 20, 37, 15, 16, 14, 25, 41, 18, 4, 38, 22, 39, 40, 24, 21, 23, 9, 29, 30, 2, 1, 6, 26, 5, 19, 13, 12, 11, 10, 8, 28, 27, 3, 7, 17, 36, 35, 31, 32]

con distancia 1900

Generacion # 108

La mejor solución es: [0, 34, 33, 20, 37, 15, 16, 14, 25, 41, 18, 4, 38, 22, 39, 40, 24, 21, 23, 9, 29, 30, 2, 1, 6, 26, 5, 19, 13, 12, 11, 10, 8, 28, 27, 3, 7, 17, 36, 35, 31, 32]

con distancia 1900

Generacion # 109

La mejor solución es: [0, 34, 33, 20, 37, 15, 16, 14, 25, 41, 18, 4, 38, 22, 39, 40, 24, 21, 23, 9, 29, 30, 2, 1, 6, 26, 5, 19, 13, 12, 11, 10, 8, 28, 27, 3, 7, 17, 36, 35, 31, 32]

con distancia 1900

Generacion # 110

La mejor solución es: [0, 34, 33, 20, 37, 15, 16, 14, 25, 41, 18, 4, 38, 22, 39, 40, 24, 21, 23, 9, 29, 30, 2, 1, 6, 26, 5, 19, 13, 12, 11, 10, 8, 28, 27, 3, 7, 17, 36, 35, 31, 32]

con distancia 1900

Generacion # 111

La mejor solución es: [0, 34, 33, 20, 37, 15, 16, 14, 18, 41, 25, 4, 38, 22, 39, 40, 24, 21, 23, 9, 29, 30, 28, 1, 6, 26, 13, 19, 5, 12, 11, 10, 8, 2, 27, 3, 7, 17, 36, 35, 31, 32]

con distancia 1896

Generacion # 240

La mejor solución es: [0, 34, 33, 20, 37, 15, 16, 14, 18, 25, 41, 23, 38, 22, 39, 24, 40, 21, 9, 8, 29, 30, 28, 4, 6, 26, 5, 19, 13, 12, 11, 10, 2, 27, 3, 1, 7, 17, 36, 35, 31, 32]

con distancia 1717

Generacion # 241

La mejor solución es: [0, 34, 33, 20, 37, 15, 16, 14, 18, 25, 41, 23, 38, 22, 39, 24, 40, 21, 9, 8, 29, 30, 28, 4, 6, 26, 5, 19, 13, 12, 11, 10, 2, 27, 3, 1, 7, 17, 36, 35, 31, 32]

con distancia 1717

Generacion # 242

La mejor solución es: [0, 34, 33, 20, 37, 15, 16, 14, 18, 25, 41, 23, 38, 22, 39, 24, 40, 21, 9, 8, 29, 30, 28, 4, 6, 26, 5, 19, 13, 12, 11, 10, 2, 27, 3, 1, 7, 17, 36, 35, 31, 32]

con distancia 1717

Generacion # 243

La mejor solución es: [0, 34, 33, 20, 37, 15, 16, 14, 18, 25, 41, 23, 38, 22, 39, 24, 40, 21, 9, 8, 29, 30, 28, 4, 6, 26, 5, 19, 13, 12, 11, 10, 2, 27, 3, 1, 7, 17, 36, 35, 31, 32]

con distancia 1717

Generacion # 244

La mejor solución es: [0, 34, 33, 20, 37, 15, 16, 14, 18, 25, 41, 23, 38, 22, 39, 24, 40, 21, 9, 8, 29, 30, 28, 4, 6, 26, 5, 19, 13, 12, 11, 10, 2, 27, 3, 1, 7, 17, 36, 35, 31, 32]

con distancia 1717

Generacion # 245

La mejor solución es: [0, 34, 33, 20, 37, 15, 16, 14, 18, 25, 41, 23, 38, 22, 39, 24, 40, 21, 9, 8, 29, 30, 28, 4, 6, 26, 5, 19, 13, 12, 11, 10, 2, 27, 3, 1, 7, 17, 36, 35, 31, 32]

con distancia 1717

Generacion # 246

La mejor solución es: [0, 34, 33, 20, 37, 15, 16, 14, 18, 25, 41, 23, 38, 22, 39, 24, 40, 21, 9, 8, 29, 30, 28, 4, 6, 26, 5, 19, 13, 12, 11, 10, 2, 27, 3, 1, 7, 17, 36, 35, 31, 32]

con distancia 1717

Generacion # 247

La mejor solución es: [0, 34, 33, 20, 37, 15, 16, 14, 18, 25, 41, 23, 39, 22, 38, 24, 40, 21, 9, 8, 29, 30, 28, 4, 6, 26, 5, 19, 13, 12, 11, 10, 2, 27, 3, 1, 7, 17, 36, 35, 31, 32]

con distancia 1710

Generacion # 248

La mejor solución es: [0, 34, 33, 20, 37, 15, 16, 14, 18, 25, 41, 23, 39, 22, 38, 24, 40, 21, 9, 8, 29, 30, 28, 4, 6, 26, 5, 19, 13, 12, 11, 10, 2, 27, 3, 1, 7, 17, 36, 35, 31, 32]

con distancia 1710

Generacion # 249

La mejor solución es: [0, 34, 33, 20, 37, 15, 16, 14, 18, 25, 41, 23, 39, 22, 38, 24, 40, 21, 9, 8, 29, 30, 28, 4, 6, 26, 5, 19, 13, 12, 11, 10, 2, 27, 3, 1, 7, 17, 36, 35, 31, 32]

con distancia 1710

Generacion # 250

La mejor solución es: [0, 34, 33, 20, 37, 15, 16, 14, 18, 25, 41, 23, 39, 22, 38, 24, 40, 21, 9, 8, 29, 30, 28, 4, 6, 26, 5, 19, 13, 12, 11, 10, 2, 27, 3, 1, 7, 17, 36, 35, 31, 32]

con distancia 1710